

An implementation of Open-RMF in an industrial environment

Michele Belletti

2025
December

Contents

I	Introduction	3
1	Background and motivation	5
2	Research questions and objectives	6
3	Scope and limitations	7
II	Literature Review	8
4	Overview of ROS1 and ROS2	10
4.1	Structure of ROS	10
4.2	Tools of ROS	12
5	Introduction to NAV2 and Google Cartographer	14
5.1	Description of Nav2	14
5.1.1	Cartographer	17
6	Review of related work on porting navigation stack from ROS 1 to ROS 2	18
7	Overview of open source fleet management systems, particularly Open-RMF	19
7.1	Open-RMF description	20

Part I

Introduction

This work presents a study on the new system, OPEN-RMF, which is still under active development but has reached a mature enough state to be deployed with robots at the Elettra Sincrotrone Trieste facility.

Chapter 1

Background and motivation

Elettra Sincontrone Trieste¹ is a major research facility that primarily provides users access to two light sources: the synchrotron Elettra and the free electron laser Fermi. In addition to operating these sources, the facility actively researches new technologies and techniques to enhance its capabilities and operations. Many recent advancements are based on robotics, which, especially after the COVID-19 pandemic and with the rapid rise of large language models (LLMs), have enabled new ways for users to interact with the infrastructure, including robot-assisted information retrieval and package delivery services. Previously, the facility developed a global planner, D-Star (D-star based optimized trajectory planner)[12], within the ROS 1 framework to enable robot navigation across large-scale environments. However, as ROS 1 has reached end-of-life, the codebase must now be migrated to ROS 2. Additionally, the facility employs robots from multiple vendors, each specialized in different tasks and not designed to coordinate with one another. In an environment that shares resources such as narrow passages and lifts, this lack of interoperability creates significant coordination challenges. To address this issue, Open-RMF² has been developed in recent years as a multiplatform system for controlling and managing heterogeneous fleets of robots.

¹<https://www.elettra.eu/>

²<https://www.open-rmf.org/>

Chapter 2

Research questions and objectives

The objectives of this thesis are as follows: to integrate a Rover Mini robot from the company RoverRobotics ¹ into the ROS 2 Nav2 ecosystem, to port the D-Star global planner from ROS 1 to ROS 2,



Figure 2.1: Rover Mini of the RoverRobotics

incorporate the robot into the Open-RMF framework for centralized fleet management, compare its performance to the ROS 1-based navigation stack, and evaluate the suitability of the Open-RMF ecosystem for deployment at the Elettra Sincrotrone Trieste facility.

¹<https://roverrobotics.com/>

Chapter 3

Scope and limitations

The principal limitation is the current lack of access to lift control systems, which prevents direct integration and operation by the robot fleet. Additionally, there is no automated system for opening and closing doors. While Open-RMF offers a high degree of customizability, and human-in-the-loop interactions can be configured—such as triggering a light or sending a notification when a robot approaches a door—this method requires a human to manually open the door or operate the lift. However, such an approach is not ideal for achieving full autonomy.

Part II

Literature Review

In this section, we provide a description of the key technologies used in this thesis. We begin by describing the Robot Operating System (ROS) and its functionality, followed by an overview of the navigation system employed, and a discussion of how the Open-RMF fleet management system operates.

Chapter 4

Overview of ROS1 and ROS2

The Robotics Operating System (ROS)¹ is a compilation of tools and libraries open-source that help the development of application in the robotic environment. otherwise from the name, it's not a Operating System (OS) but an Software Development Kit (SDK) that gives at the user all the tools for creating your robot giving some typically functionality of an OS: hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management. The ROS ecosystem is based on:

- A communication called middleware (**RMW**) or plumbing made with **DDS** (Data Distribution Service). This allow the Nodes to share information between each other with an anonymous publish/subscribe pattern allowing fault isolation, separation of concerns and clear interfaces and a synchronous system (remote procedure call)RPC-style communication between services. The default one is eProsima Fast DDS (`rmw_fastrtps_cpp`), but we are going to use Eclipse Cyclone DDS (`rmw_cyclonedds_cpp`) that has better compatibility with Nav2.
- Tools to control and improve the development of the software like launch, introspection, debugging, visualization, plotting, logging, and playback.
- Provides tools and libraries for obtaining, building, writing, and running code across multiple computers.
- A active and big community with at the center the Open Robotics².

4.1 Structure of ROS

The documentation³ give all the information of ROS, some functionality write below are not available for ROS 1:

- The **ROS Graph** is a peer-to-peer network of processes that elaborated simultaneously and communicate with each other.

¹<https://www.ros.org/>

²<https://www.openrobotics.org/>

³<https://docs.ros.org/en/rolling/index.html>

- **Nodes** is a modular process and should be a specific resolution for a problem to be more easily communicate with other Nodes through topics, services, action and parameters. They also can be based on **LifecycleNode** that contain a state machine transitions for bringup and teardown to allow more deterministic behavior.

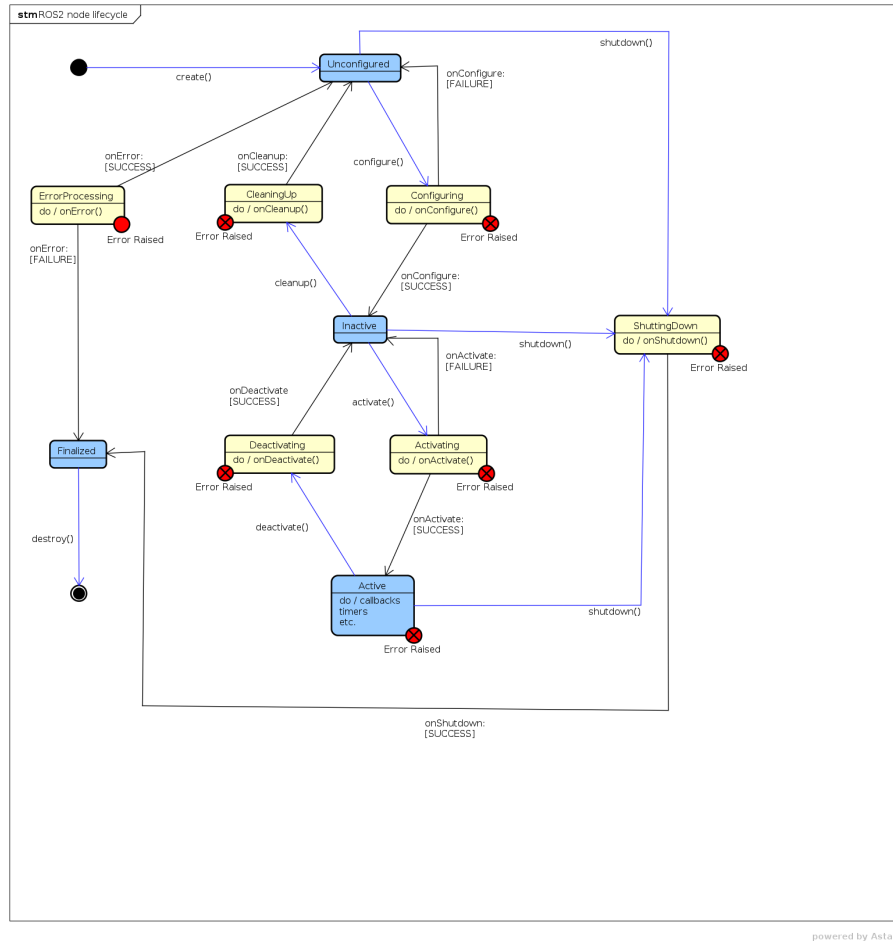


Figure 4.1: State Machine of LifecycleNode

Then can interact with other nodes with a **Composition**, that enable improve performance bypassing the middleware[4].

- **Topics** are the bus for communication between Nodes to share a streams of messages in a publisher to subscriber structure.

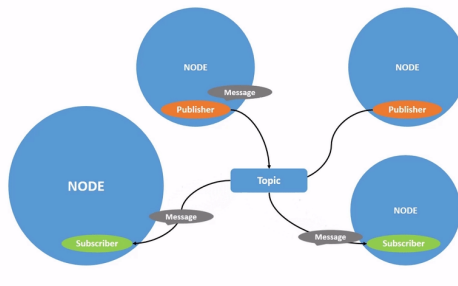


Figure 4.2: Topic process

- **Service** are based on the request and response model. When a node with a Service client send a Request Message to Node with a Service Server it will send a Response Message to that specific Service Client.

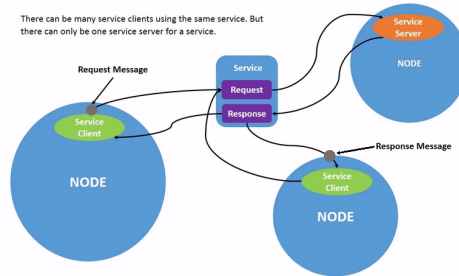


Figure 4.3: Service process

- **Parameters** are configuration of a node and allow to modified them without changing the code.
- **Actions** were created to fulfill the needs of something for tracking long task. They are a combination of two service (goal and result) and a pub-sub (feedback). It send a goal request and receive a response then send a result request, the node then start to pub to the feedback topic and then response to the result request.

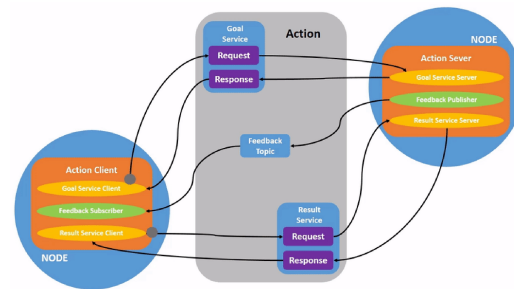


Figure 4.4: Action process

4.2 Tools of ROS

Now i will describe the tools that are built in ROS for developing robotics system:

- **Launch** is a file structure that allow to start multiple nodes with a set of parameters.
- **rqt** is a graphical user interface (GUI) tool for ROS 2. With a series of plugins allow to interact with an interface for helping in the development, in the monitoring and diagnostic the ROS system.
- **rqt_console** visualize the log generate by the ROS 2 system.
- **Rviz** is a 3D visualizer for the Robot Operating System framework?
- **rosdep** is a dependency installer utility.

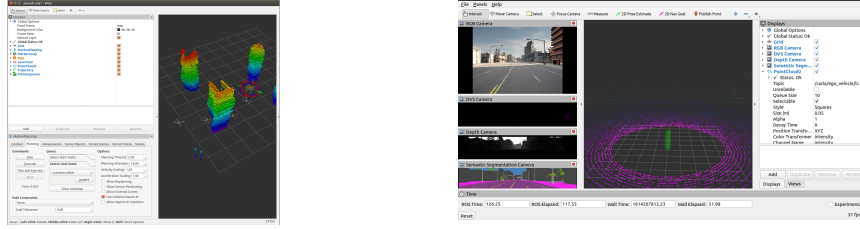


Figure 4.6: Examples of visualization of ROS data with Rviz

- **sros2** is a package that provide the tools and instruction to apply security on DDS.

Another tools that are connected with ROS and helps develop is the simulation environments. They allow to test the code on a simulated robot in different condition to find possible bugs in the code that if there was launched on a real robot can be harmful to it or the environment. They provide realistic physical mechanics at all the parts of simulation and in the documentation of ROS there are listed 2: Gazebo and Webots. The first one is also a project under the Open Robotics and for a lot of projects is the default used like the one we will use. Gazebo offer:

- **ROS Integration** a bridge to converts the protobuf messages to ROS.
- **Performance tools** that allow to have it distributed on servers, dynamically asset loading and control of the time step for simulation to have the best performances.
- **Realistic simulation** that allow to insert a variety of sensor with their noise in a 3D rendering word built on Ogre 2.1⁴ and with the physic engine DART⁵.
- **Extensibility** because its built on modularity, it allow with plugins to run code that interact with the simulation.

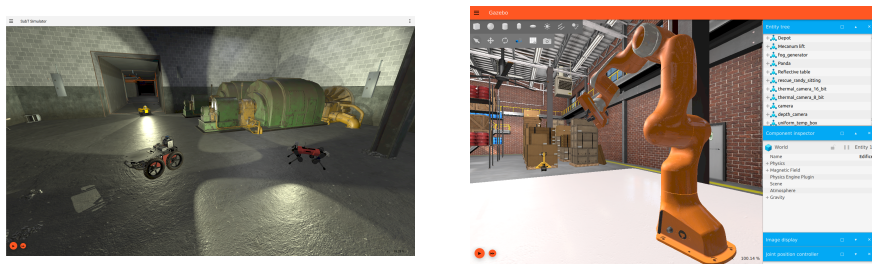


Figure 4.8: Examples of Gazebo

⁴<https://wiki.ogre3d.org/Home>

⁵<https://dartsim.github.io/>

Chapter 5

Introduction to NAV2 and Google Cartographer

Multiple tools are available for move a robot in an environment. The one that give a full system is Nav2 ¹ [7] the successor of ROS Navigation Stack of ROS 1.

5.1 Description of Nav2

Nav2 gives all the tools (perception, planning[9], control, localization, visualization) for whichever robot to navigate autonomously and reliably also thought hard environment. It allow to build a model of the environment from the data of the sensors, calculated dynamically the best path to follow to avoiding obstacles using a map, set the velocity of the motors and allow to follow logic step using a behaviors tree.

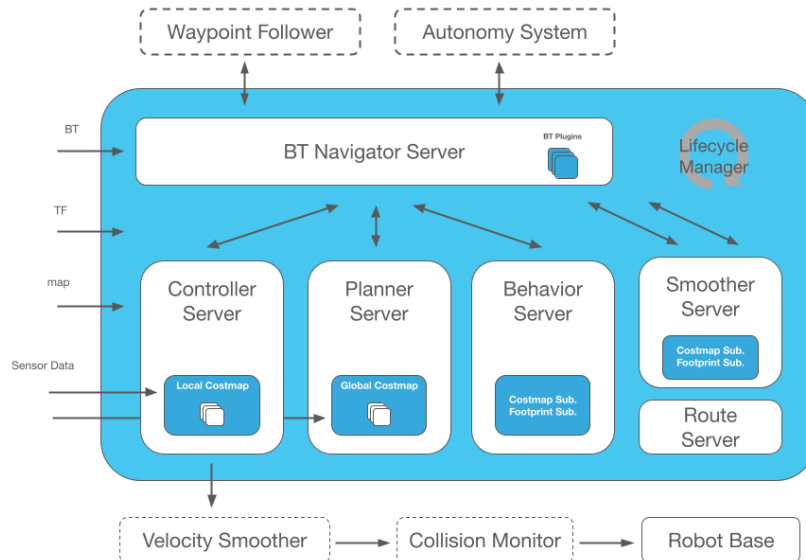


Figure 5.1: Nav2 architecture

These are also some of the updates that have changed from ROS 1 other less notable changes are the use of:

¹<https://nav2.org/>

- **Behavior trees**^[1]: gives to Nav2 a structure for completing a series of tasks in a human readable way. Their structure is similar to a tree and allow to create complex system that can be tested with different tools. The package used for this is BehaviorTree.CPP² that respect others allow to link different tree.
- **nav2_util LifecycleNode** is a wrapper of rclcpp_lifecycle LifecycleNode of the standard structure of ros as described in Nodes 4.1 and add a bond connection to the **Lifecycle Manager**, node that knows the states of each node, and allow to transition down its lifecycle nodes for safety if some node has crashed.
- **Action Server**: the functionality are described in the structure of ROS 4.1 and they are implemented in Nav2 for the interaction with the Behavior Tree (BT) like NavigationToPose (move to a location), NavigationThroughPoses (allow to select multiple waypoints), etc.

The control is based on this nodes:

- **Controller Server** is the node that handle the request for moving the robot, using tree plugins progress checker (check if the robot has make some progress in following the path), goal checker (check if the robot has reached the goal position) and controller (generate the velocity command using the **local costmap** to move the robot avoiding collision)
- **Planner Server** is the node that handle the request for calculate the path, using a loaded plugin planner, to reach a position using the **global costmap** and the data from sensors.
- **BT Navigator Server** is a Behavior Tree-based implementation of navigation that implement the NavigateToPose, NavigateThroughPoses, and other task interfaces to increment the flexibility during navigation.
- **Smoother Server** is the node that handle the request for smoothing, with plugins, a path minimizing sharp turns and rapid rotations from the path.
- **Behavior Server** is the node that handles the behavior request like: spin, backup, wait.
- **Waypoint Follower** is a way/example for controlling the robot for doing an orchestrated task. This implementation take a list of waypoints, call the NavigateToPose action server and at each waypoint do a task (wait, take a photo, ...) as a plugin. This server can be changed with any **Autonomy System**
- **Route Server** for computing routes through a predefined navigation graph rather than using freespace planning like the Planner Server.
- **Velocity Smoother** as the name say smooth the velocity, acceleration and deadbands sent by Nav2 to the robot.

²<https://www.behaviortree.dev/>

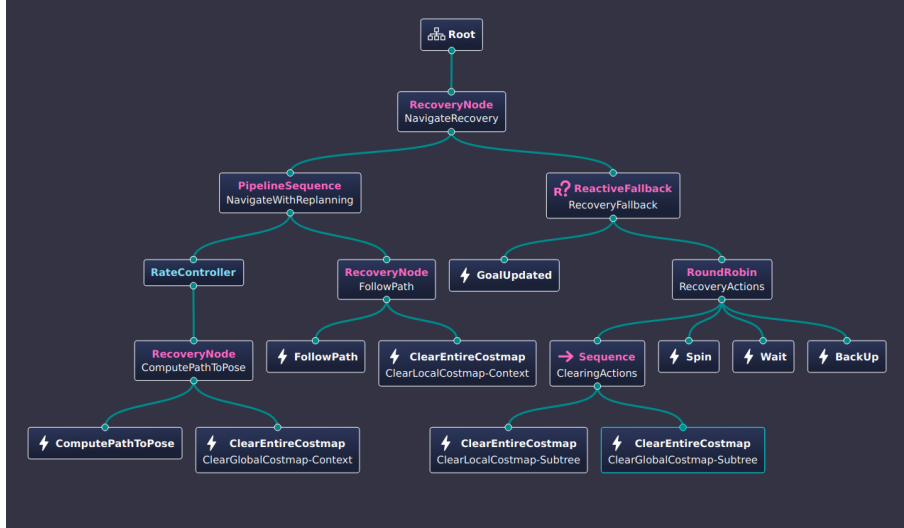


Figure 5.2: Humble Nav2 BT navigation `navigate_to_pose_w_replanning_and_recovery.xml` loaded with Groot2

- **Costmap** is a representation of the environment using a regular 2D grid of cells with a cost (unknown, free, occupied, or inflated) and is used for generate the global plan or to compute local control efforts.

The next things that allow to use Nav2 is a:

- **state estimation** of the robot that generate a suitable TF tree, a tree structure of coordinate frames generated using the transform library[2], based on the REP 105, that is should provide this structure: `map` $\rightarrow$ `odom` $\rightarrow$ `base_link`. According to the REP 105 they are defines as:
 - `map` : a world fix frame that is not continuous that is the pose of the robot can change in discrete jumps at any time. This is useful as a long-term global reference but not in the short-term.
 - `odom` : a world fix frame that can drift over time to be continuous. This allow to be useful in an short-term local reference but not good for long-term one.
 - `base_link`: a robot fix frame.

The `odom` $\rightarrow$ `base_link` link can be calculated using different sensors. To increase the performance of our estimation we will use the `robot_localization` a node that provide nonlinear state estimation nodes using Kalman Filter [8]. The `map` $\rightarrow$ `odom` link can be generate with a global positioning system like Simultaneous Localization And Mapping (SLAM) method or using the one provided by Nav2, Adaptive Monte-Carlo Localization (AMCL).

- **map** that can be build during navigation using a SLAM tool or loaded. A powerful SLAM is Cartographer that will be described in the next section and it's the one used to build the map that we are going to use.

5.1.1 Cartographer

Cartographer[3] is a SLAM that work using two subsystem: a **Local SLAM** (called **Frontend**) that build a series of locally consistent **submaps** from successive scans and the pose extrapolator but with the possibility that can drift during acquisition and a **Global SLAM** (called **Backend**) that find loop closure constraints scan-matching scans against submaps using a global optimization in a way to obtain a consistent map. It can perform a 2-3 cm accuracy mapping but required a lot tuning of the parameters.

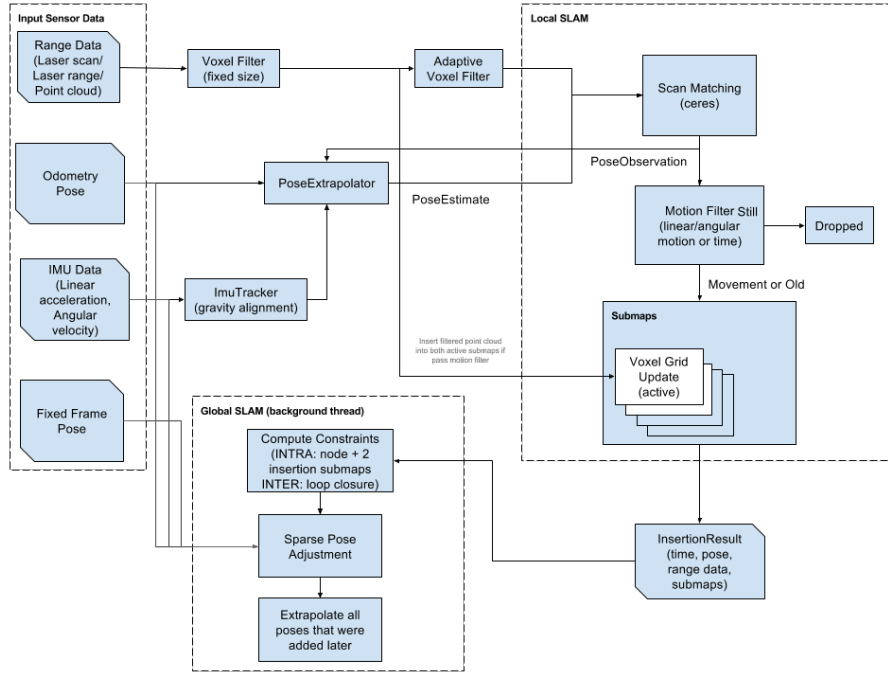


Figure 5.3: Cartographer Algorithm

Chapter 6

Review of related work on porting navigation stack from ROS 1 to ROS 2

The big diffusion of ROS as a software for controlling robots bring new users cases that were not considered when was built. Starting as an academic project, the new cases that has met are the need of a way to control a fleet of them, deveoping on little embedded board and microcontrollers, real time system, not ideal network and the code was not designed for industrial usage. These changes made impossible to save ROS from a total rewrite of the code that brigh to have ROS 2, all more documentation is accessible at <https://design.ros2.org/> . Multiple company started using the ROS 2 ecosystem and how it increase their development[6] and also in the Nav2[9]. New projects are born like micro-ROS ¹ for controlling microcontrollers. And the new tool Node Composition allow an higher degree of performance optimization [4] and also allow for a better security[11] While more detailed information about how to change the code are described in these sources, like the official documentation² and also secondary ³ and are the tools that are going to be used in the porting of the global planner. **The notable changes, also described in that paper are:**

Like describe in this paper the use of ROS 2 respect ROS 1 bring many advantages and is the way to go.

¹<https://micro.ros.org/>

²<https://docs.ros.org/en/rolling/How-To-Guides/Migrating-from-ROS1.html>

³https://industrial-training-master.readthedocs.io/en/melodic/_source/session7/ROS1-to-ROS2-porting.html

Chapter 7

Overview of open source fleet management systems, particularly Open-RMF

Currently these are some the fleet management system available:

- **Isaac Mission Dispatch**¹ is a NVIDIA microservice enable fleet management software to submit missions to multiple robots and monitor the robot and mission status. It provides a connection between the robots and the fleet management system but does not handle logistics such as task allocation or conflict resolution, e.g., robots with intersecting paths. The implementation relies VDA5050 protocol as an industry standard between a cloud control service and mobile robots, and uses MQTT as a lightweight, publish-subscribe, machine to machine network protocol designed for devices with resource constraints and limited network bandwidth.
- **ROOSTER Fleet Management**² is a ROS 1 based open source project to develop a heterogeneous fleet management solution with task allocation, scheduling and routing capabilities.
- **Open Robotics Middleware Framework (Open-RMF)**³ is a free, open source, modular software system that enables sharing and interoperability between multiple fleets of robots and physical infrastructure, like doors, elevators and building management systems.
- **Robofleet**⁴[10] based on ROS 1 is open-source system which provides inter-robot communication, remote monitoring, and remote tasking for a heterogeneous fleet of ROS-enabled service-mobile robots that is designed with the practical goals of resilience to network variance and security control in mind.

While others gives an infrastructure for start the control of a fleet:

- **Transitive Robotics**⁵ is an open-source framework for full-stack robotics. It is designed to make it easy to create capabilities that seamlessly connect

¹https://github.com/NVIDIA-ISAAC/isaac_mission_dispatch

²<https://github.com/ROOSTER-fleet-management>

³<https://www.open-rmf.org/>

⁴<https://github.com/ut-amrl/robofleet>

⁵<https://transitiverobotics.com/>

robot, cloud, and web front-ends including a dynamically updating shared data model. It offers solutions to many problems developers encounter when building full-stack robotic applications, including unreliable networks, robots that are regularly offline or turned off, as well as cross-device version.

- **Robot Web Tools**⁶ is a collection of tools that allow to communicate with a remote robot.

Given that Open-RMF gives a control over multiple fleet, a structure for them to interact with each other to obtain the best outcome, a lot of tool to build the system, is based on ROS 2 and is open source, it is going to be the base on witch we will build our fleet. Now we will all the capabilities of Open-RMF

7.1 Open-RMF description

Open-RMF is a flexible tool set build on ROS 2 for allowing the interaction between heterogeneous robot fleets. It uses a standardized protocol for communication with infrastructures, environments and automations to optimize the use of the critical resources (like other robots, lifts, doors, narrow passages) of a system to control the traffic. It gives intelligence to a system avoiding conflicts for the available resources and assigned it correctly. This figure describe how works Open-RMF:

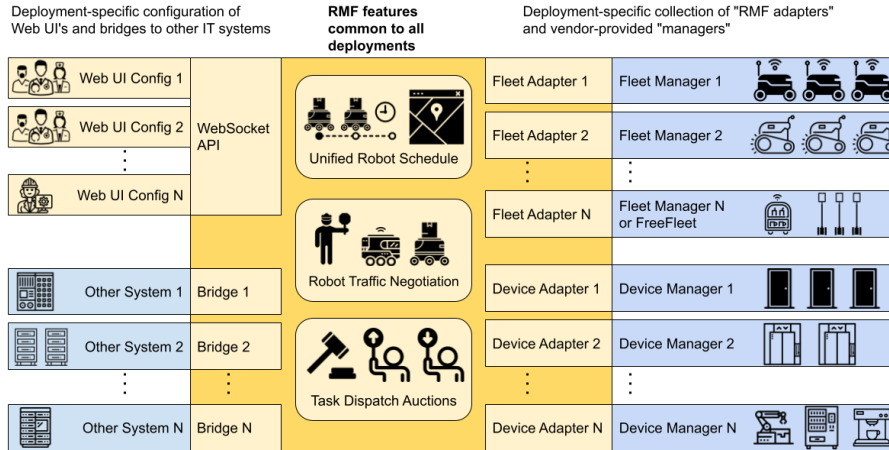


Figure 7.1: Open-RMF Structure

There are 3 type of fleet adapter :

- *Full Control*: have full status updates and full control on the path
- *Traffic Light*: have full status updates and a pause/resume control
- *Read Only*: have only status updates. It can be only one of this type because more of them will make avoiding deadlock or resource conflict nearly impossible.

In case there is a fleet with *No Interface* this can result deadlocks on the available resources and is not compatible with a RMF system.

The Open-RMF core `rmf_core` is built on this list of packages:

⁶<https://robotwebtools.github.io/>

- **rmf_traffic**: Core scheduling and traffic management systems based on the following traffic deconfliction: *Prevention* and *Negotiation* using a living platform-agnostic Traffic Schedule Database.

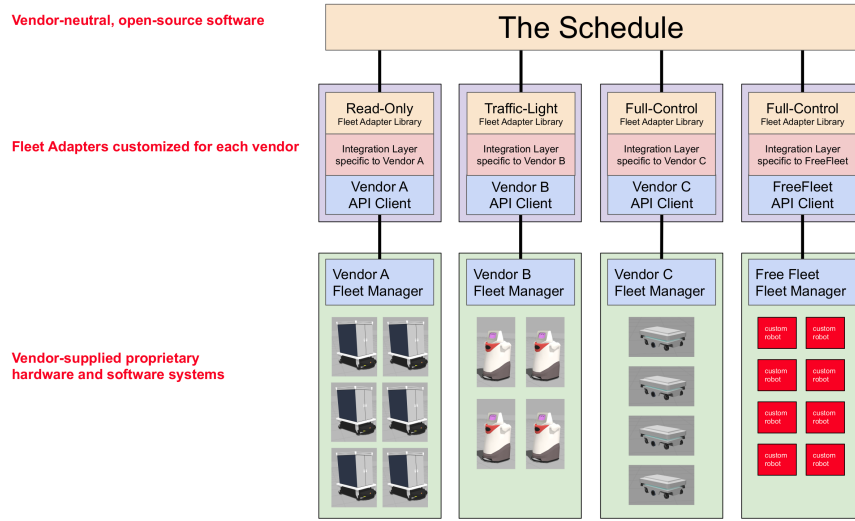


Figure 7.2: Open-RMF Schedule structure

- *Prevention* when there are some changes in the DB, like cancellations, the fleet manager can plan the routes to avoid conflicts.
- *Negotiation* When thought the Database there is an upcoming conflict between two or more schedule participants, it will send a conflict notice out to the relevant fleet managers, that will reply with their preferable path that can accommodate the others and through a external judge choose the best path to follow

Conflict avoidance for automated guided vehicle AGV's is implemented with time-dependent extension to A* search for both path and task. This search takes time into account so that it can find paths through space and time that account for the motions of other agents that are in the traffic schedule. It's not designed for autonomous mobile robot (AMR) that is in development.

- **rmf_traffic_ros2**: provide the interface between rmf_traffic and ros2
- **rmf_task**: APIs and base classes for defining and managing Tasks in RMF. RMF supports three types of task requests out of the box:
 - *Clean*: For robots capable of cleaning floor spaces in facilities
 - *Delivery*: For robots capable of delivering items between locations in facilities
 - *Loop*: For robots capable to navigating back and forth between locations in facilities
- **rmf_dispatcher_node**: RMF simplifies task allocation and management across multi-fleet systems. When a user submits a new task request, RMF will intelligently assign it to the robot in the fleet that can best perform the task as described in the image.



Going once, going twice, sold!

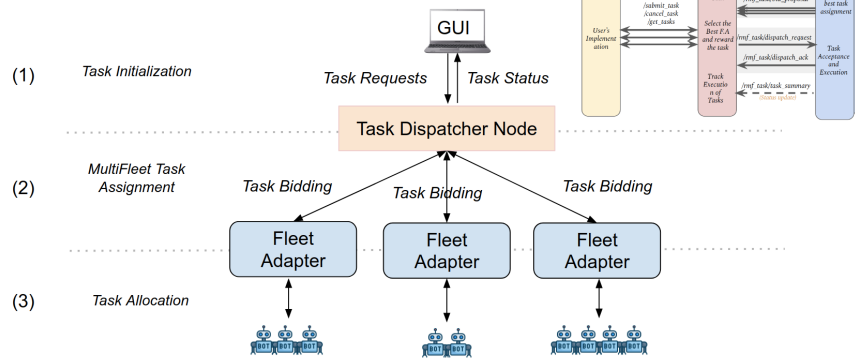


Figure 7.3: Open-RMF Bid Task

- `rmf_battery`: rmf battery estimation
- `rmf_ros2`: ros2 adapters and nodes and python bindings for `rmf_core`
- `rmf_utils`: utility for rmf

For the development these are the tools ready to use to develop:

- **RMF demos** contains demonstrations of the capabilities and are a starting point for the development.
- **Traffic Editor** is a graphical editor that allow to create a map with traffic patterns and the place of interaction that RMF will use to control the traffic. It also can generate a Gazebo world for simulation.

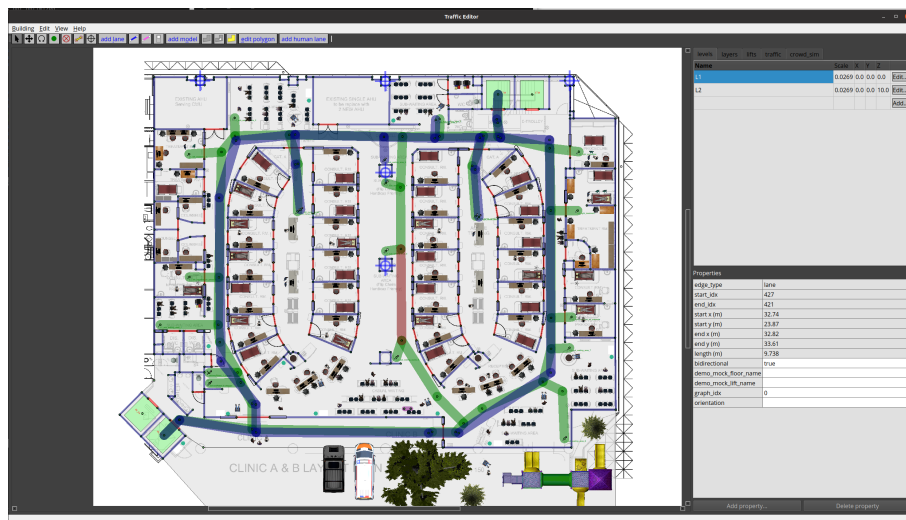


Figure 7.4: Traffic Editor

- **Free Fleet** is a free fleet adapter allow to integrate standalone mobile robots into Open-RMF to form a heterogeneous fleet.
- **RMF Schedule Visualizer** is a plugin for rviz to see and control Open-RMF.

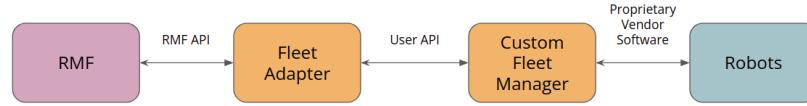


Figure 7.5: Fleet Adapter

- **RMF Web UI** is a web application for visualization and control the Robotic Middleware that is `rmf_core`

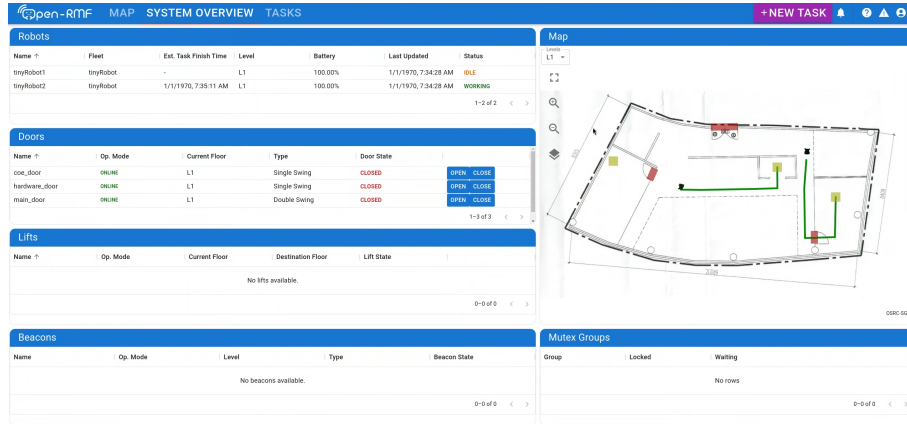


Figure 7.6: RMF Web

- **RMF Simulation** plugins to simulate each capability in gazebo.
- **Simulation assets** are the models freely available.

today the tool for converting the data are <https://www.ros.org/blog/noetic-eol/>
implement also planning benchmark

describe and **use** composition in nav2 to obtain better performance [5]

Inoltre per gestire il traffico sulla rete utilizzeremo zenoh (Zero Overhead Network Protocol) ed il bridge per ros2 zenoh-bridge-ros2dds che è pub/sub/query protocol unifying data in motion, data at rest and computations. It elegantly blends traditional pub/sub with geo distributed storage, queries and computations, while retaining a level of time and space efficiency that is well beyond any of the mainstream stacks. data liberator protocol. Zenoh liberates data in several dimensions.

with chatgpt possiamo gestire il robot in due modi: si ferma ed esegue l'azione non comunicando con rmf (la linea rimane bloccata dal suo passaggio)

Bibliography

- [1] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI*. July 2018. DOI: 10.1201/9780429489105. URL: <http://dx.doi.org/10.1201/9780429489105>.
- [2] Tully Foote. “tf: The transform library”. In: *Technologies for Practical Robot Applications (TePRA), 2013 IEEE International Conference on*. Open-Source Software workshop. 2013, pp. 1–6. DOI: 10.1109/TePRA.2013.6556373.
- [3] Wolfgang Hess et al. “Real-Time Loop Closure in 2D LIDAR SLAM”. In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1271–1278.
- [4] Steve Macenski et al. *Impact of ROS 2 Node Composition in Robotic Systems*. 2023. arXiv: 2305.09933 [cs.R0]. URL: <https://arxiv.org/abs/2305.09933>.
- [5] Steve Macenski et al. “Impact of ROS 2 Node Composition in Robotic Systems”. In: *IEEE Robotics and Automation Letters* 8.7 (2023), pp. 3996–4003. DOI: 10.1109/LRA.2023.3279614.
- [6] Steven Macenski et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [7] Steven Macenski et al. “The Marathon 2: A Navigation System”. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020.
- [8] T. Moore and D. Stouch. “A Generalized Extended Kalman Filter Implementation for the Robot Operating System”. In: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*. Springer, 2014.
- [9] DV Lu A. Merzlyakov M. Ferguson S. Macenski T. Moore. “From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2”. In: *Robotics and Autonomous Systems* (2023).
- [10] Kavan Singh Sikand et al. *Robofleet: Open Source Communication and Management for Fleets of Autonomous Robots*. 2021. arXiv: 2103.06993 [cs.R0]. URL: <https://arxiv.org/abs/2103.06993>.
- [11] Victor Mayoral Vilches et al. *SROS2: Usable Cyber Security Tools for ROS 2*. 2022. arXiv: 2208.02615 [cs.CR]. URL: <https://arxiv.org/abs/2208.02615>.

-
- [12] Andrey Vukolov et al. “Universal Configurable Navigation and Control System for Industrial Unmanned Ground Vehicles with Differential Chassis”. In: *Advances in Mechanism and Machine Science*. Ed. by Masafumi Okada. Cham: Springer Nature Switzerland, 2023, pp. 287–301. ISBN: 978-3-031-45770-8.